

Kotlin Multiplatform Cheat Sheet

Baseline: Kotlin 2.4.20, Compose Multiplatform 1.12.1, AGP 9.4, Xcode 26.4, Ktor 3.6.0, SQLDelight 2.4.0, Koin 4.2.2 (as of 2026-09-27). Examples use the TrailMate Everywhere project. Quick reference only — verify against the official docs before production use.

1 · What to share & stability

Logic only: shared Kotlin + native UI (SwiftUI, Jetpack Compose). **Logic + UI:** Compose Multiplatform (CMP) everywhere.

Platform	KMP	CMP
Android, JVM/desktop	Stable	Stable
iOS	Stable	Stable
Web Kotlin/JS	Stable	—
Web Kotlin/Wasm	Beta	Beta
watchOS, tvOS	Beta	—

Baseline: Kotlin 2.4.20, CMP 1.12.1, AGP 9.4, Xcode 26.4. Swift export & SwiftPM import: **Alpha**.

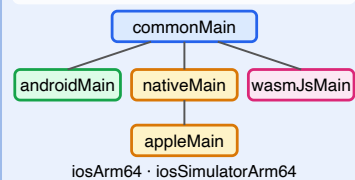
2 · Compilers, targets & tiers

Kotlin/JVM (JVM bytecode, Android) · **Kotlin/Native** (LLVM, native binaries) · **Kotlin/JS** (JS, ES2015+) · **Kotlin/Wasm** (WebAssembly, Beta).

Tier 1: macosArm64, iosSimulatorArm64, iosArm64. **Tier 2:** Linux, watchOS, tvOS Arm. **Tier 3:** iosX64, mingwX64, androidNative*. **Deprecated:** macosX64, tvosX64, watchosX64, watchosArm32. Min OS: iOS 15, macOS 12. Apple targets build only on macOS with Xcode.

3 · Layout & source sets

```
sharedLogic/src/
commonMain/ commonTest/
androidMain/ iosMain/
appleMain/ nativeMain/ (auto)
sharedUI/ androidApp/ iosApp/
desktopApp/ webApp/
```



Default hierarchy template creates intermediate sets (appleMain, nativeMain); dependsOn by hand is legacy.

4 · kotlin {} DSL & version catalog

```
// sharedLogic/build.gradle.kts
plugins {
    alias(libs.plugins.kmp)
    alias(libs.plugins.androidKmpLib)
}
kotlin {
    android {
        namespace = "com.example.shared"
        compileSdk = 37; minSdk = 26
    }
    iosArm64(); iosSimulatorArm64()
    jvm("desktop")
    wasmJs { browser() }
    sourceSets.commonMain {
        dependencies {
            implementation(libs.ktor.core)
        }
    }
}
# gradle/libs.versions.toml
[versions]
kotlin = "2.4.20"
ktor = "3.6.0"
app = "9.4.1"
[plugins]
androidKmpLib = { id = "com.android.kotlin.multiplatform.library", version.ref = "app" }
```

Compatibility (2.4.20): Gradle 7.6.3–9.7.0, AGP 8.5.2–9.3.1 tested, Xcode 26.4.

5 · Android-KMP plugin & AGP 9

Shared modules use `com.android.kotlin.multiplatform.library` (AGP 8.10+), configured in `kotlin { android { ... } }`. No `com.android.library` in shared modules. Under AGP 9 `com.android.application` cannot combine with the KMP plugin: the app lives in its own **androidApp** module. Built-in Kotlin: no `kotlin-android` plugin. Legacy variant API stays until AGP 10.

6 · expect/actual vs interfaces

```
// commonMain
expect fun platformName(): String
// iosMain
actual fun platformName() = "iOS"
// commonMain: interface + DI
interface LocationProvider {
    suspend fun current(): Location
}
```

Prefer an **interface + DI** for services; use `expect/actual` for functions, constants and small classes (`expect/actual` classes are Beta and need `-Xexpect-actual-classes`).

7 · Dependency ecosystem

Add libraries per source set: `commonMain.dependencies { implementation(...) }`. Discover libraries on **klibs.io**. **KSP** per target: `kspAndroid`, `kspIosArm64`, `kspCommonMainMetadata`. **cinterop** wraps C/Obj-C via `.def` files. **SwiftPM import** (Alpha): `swiftPMDependencies { }`. **CocoaPods:** Stable, not deprecated.

8 · Coroutines & dispatchers

Dispatcher	Android/JVM	iOS	Wasm/JS
Main	main loop	main queue	browser thread
Default	CPU pool	CPU pool	same thread
IO	64-thread pool	64-worker pool	alias of Default

Desktop Main needs `kotlinx-coroutines-swing`. `StateFlow` for state, `SharedFlow` for events; `stateIn(scope, WhileSubscribed(5_000), init)`. New memory manager: no `freeze()`.

9 · Shared ViewModel

```
// TrailListViewModel.kt
class TrailListModel(
    repo: TrailRepository
) : ViewModel() {
    val state: StateFlow<UIState> =
        repo.trails()
            .map(::Content)
            .stateIn(
                viewModelScope,
                WhileSubscribed(5_000),
                Loading()
            )
}
```

`androidx.lifecycle:lifecycle-viewmodel 2.11.0` is multiplatform. Expose DTOs/immutable state to Swift.

10 · Ktor client

```
HttpClient(engine) {
    install(ContentNegotiation) {
        json()
    }
    install(HttpTimeout) {
        requestTimeoutMillis = 15_000
    }
}
// OkHttp: Android; Darwin: iOS
// Java|CI0: desktop; Js: web
```

Ktor 3.6.0 + `ktor-serialization-kotlinx-json`; test with `MockEngine`.

11 · SQLDelight · Room 3 · DataStore

SQLDelight 2.4.0 (app, cash, sqldelight): `.sq` files → typed Kotlin, drivers per platform, `asFlow()`. **Room 3** (androidx.room3 3.0.3): `@Database, @ConstructedBy`, bundled SQLite driver, KSP per target. **DataStore** 1.2.1 for prefs (createWithPath). multiplatform-settings for simple keys.

12 · Koin & DI

```
val trailMateModule = module {
    single<TrailRepository> {
        OfflineFirstTrailRepository(
            get(), get()
        )
    }
    viewModelOf(::TrailListModel)
}
startKoin {
    modules(trailMateModule)
}
```

Koin 4.2.2 (`koin-compose-viewmodel`). Alternatives: Metro 1.4.5, `kotlin-inject` 0.9.0, manual DI.

13 · iOS integration options

Option	Use when	Status
Direct (embedAndSig n...)	one repo, Xcode build phase	Stable
SwiftPM export	ship XCFramework as a package	Stable
SwiftPM import	use Swift packages in Kotlin	Alpha
CocoaPods	existing Pods setup	Stable

Package binaries as **XCFrameworks** (no bitcode).

14 · Obj-C/Swift mapping, SKIE, Swift export

Obj-C header: `suspend` → completion handler, `Flow` → opaque `Flow`, sealed class → class hierarchy, `enum` → class, `Int` → `Int32`, `Long` → `Int64`, top-level fun → `FileKt` object. **SKIE** 0.10.15: `suspend` → `async`, `Flow` → `AsyncSequence`, exhaustive enums. **Swift export** (Alpha): direct Swift API, no Obj-C header. Obj-C/C interop: Beta.

15 · suspend & Flow from Swift

```
// SwiftUI, with SKIE
.task {
    for await s in vm.state {
        self.ui = s
    }
}
try await repo.refresh()
// KMP-NativeCoroutines 1.0.6
asyncSequence(for: vm.stateFlow)
```

Cancel by cancelling the Swift Task; keep UI on `@MainActor`.

16 · Kotlin/Native memory & binaries

New memory manager is default (1.7.20); freezing removed (1.9.20). **CMS GC** default since 2.4.0. Binary options: `-Xbinary=bundleId=...`, `binaryOption("...")`, `isStatic = true`, `baseName`. Export only needed API; `dSYM` for crashes; `linkReleaseFrameworkIosArm64` for size.

17 · CMP entry points

```
// Android
setContent { App() }
// iOS
fun MainViewController() =
    ComposeUIViewController { App() }
// Desktop
application {
    Window { exitApplication,
        title = "TrailMate" } { App() }
}
// Web (wasmJs)
ComposeViewport(document.body!!) { App() }
```

Compose compiler plugin: `org.jetbrains.kotlin.plugin.compose`. CMP min: Android API 21, iOS 14. Hot Reload 1.2.0 (desktop).

18 · CMP resources

`composeResources/` with `drawable`, `font`, `values/strings.xml`, `files`; `qualifiers -de, -night, -xhdpi`. Use generated `Res.string.title`: `Text(stringResource(Res.string.trails))` `Image(painterResource(Res.drawable.logo), null)`

19 · CMP navigation

Navigation Compose (type-safe @Serializable routes) from `org.jetbrains.androidx.navigation 2.9.2; Navigation 3` (navigation3-ui 1.1.2). Deep links per platform; `ViewModel` via `viewModel { } / koineViewModel()`. Lifecycle ports: `org.jetbrains.androidx.lifecycle 2.11.0`.

20 · CMP on iOS & interop

`UIKitView` embeds `UIKit` (e.g. `MKMapView`); `UIKitViewController` a controller; `ComposeUIViewController` embeds `Compose` in `SwiftUI` via `UIViewControllerRepresentable`. Touch and accessibility mapped to `UIKit`. Liquid Glass supported.

21 · Desktop packaging

```
compose.desktop {
    application {
        mainClass = "MainKt"
        nativeDistributions {
            targetFormats(Dmg, Msi, Deb)
            packageName = "TrailMate"
        }
    }
}
// ./gradlew
// packageDistribution
// ForCurrentOS
```

macOS needs signing + notarization; `createDistributable`, `runDistributable`.

22 · Wasm & JS

```
wasmJs { browser() } (Beta) for CMP web;
js(IR) { browser() } for Kotlin/JS
(Stable). JS interop: @JsExport, external, JsAny. Run: wasmJsBrowserDevelopmentRun; production: wasmJsBrowserDistribution. Needs a modern browser with WasmGC.
```

23 · Testing

`kotlin("test")` in `commonTest`; **fakes over mocks**; **Mokkery** 3.5.0 for KMP mocking (`MockK` is JVM only); **Turbine** 1.2.1 for `Flow`; `runTest` from `coroutines-test`; `Ktor MockEngine`; `CMP runComposeUiTest` (Experimental). Run: `allTests`, `iosSimulatorArm64Test`.

24 · Publishing

```
plugins {
    id("com.vanniktech.maven.publish")
}
mavenPublishing {
    publishToMavenCentral()
    signAllPublications()
}
// SwiftPM: XCFramework zip
// + Package.swift binaryTarget
```

`vanniktech 0.37.0` (Central Portal), `Dokka 2.2.0`, `abiValidation()` (Experimental), `npm-publish` for JS. Apps: `Play (AAB)`, `App Store` (privacy manifest), desktop installers.

25 · CI (GitHub Actions)

`runs-on: macos-latest`
`steps:`
– `uses: actions/checkout@v7`
– `uses: actions/setup-java@v6`
– `uses: gradle/actions/setup-gradle@v6`
– `uses: maxim-lobanov/setup-xcode@v1`
with: `{xcode-version: "26.4"}`
– `run: ./gradlew allTests build`

`Pin Xcode 26.4 for Kotlin 2.4.20; macOS runner for iOS targets.`

26 · Adoption checklist

1. Pick a path: logic-only or shared UI
2. Extract one feature (e.g. networking) into `sharedLogic`
3. Move Android to `androidApp` (AGP 9)
4. Wire iOS via direct integration or `SwiftPM`
5. Add `SKIE`, tests and CI early
6. Mono-repo for one team; multi-repo + `XCFramework` for separate teams